

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6

«Введение в СУБД MongoDB. Работа с БД в СУБД
MongoDB»

Обучающийся: Лютый Никита Артемович

Факультет: Прикладной информатики

Группа: К3240

Преподаватель: Говорова Марина Михайловна

Санкт-Петербург

2026

Содержание

1	Цель работы	2
2	Краткие теоретические сведения	2
3	Ход работы	3
3.1	Установка и проверка MongoDB	3
3.2	Создание и удаление базы данных в лабораторной работе 6.1	5
3.3	Заполнение коллекции unicorns	7
3.4	Выборка данных	8
3.5	Логические операторы	11
3.6	Работа с вложенными документами	13
3.7	Использование JavaScript и курсоров	14
3.8	Агрегированные запросы	15
3.9	Редактирование документов	16
3.10	Удаление документов	20
3.11	Ссылки между документами	22
3.12	Индексы	24
3.13	План запроса	25
4	Ответы на контрольные вопросы	26
5	Вывод	28

1 Цель работы

Цель лабораторной работы 6.1 — получить практические навыки установки СУБД MongoDB и начальной работы с базами данных, коллекциями и документами. Цель лабораторной работы 6.2 — получить практические навыки выполнения CRUD-операций, работы с вложенными объектами, курсорами, агрегированными запросами, изменением данных, ссылками и индексами в MongoDB.

2 Краткие теоретические сведения

MongoDB является документо-ориентированной NoSQL СУБД. Данные хранятся не в таблицах, а в коллекциях, состоящих из документов. Документ представляет собой набор пар “ключ–значение” и может содержать простые значения, массивы, вложенные документы и массивы вложенных документов.

Для хранения данных MongoDB использует формат BSON — бинарное представление JSON-подобных документов. В каждом документе присутствует уникальное поле `_id`. Если пользователь не задает его явно, MongoDB создает идентификатор автоматически.

Основные команды и методы, использованные в работе:

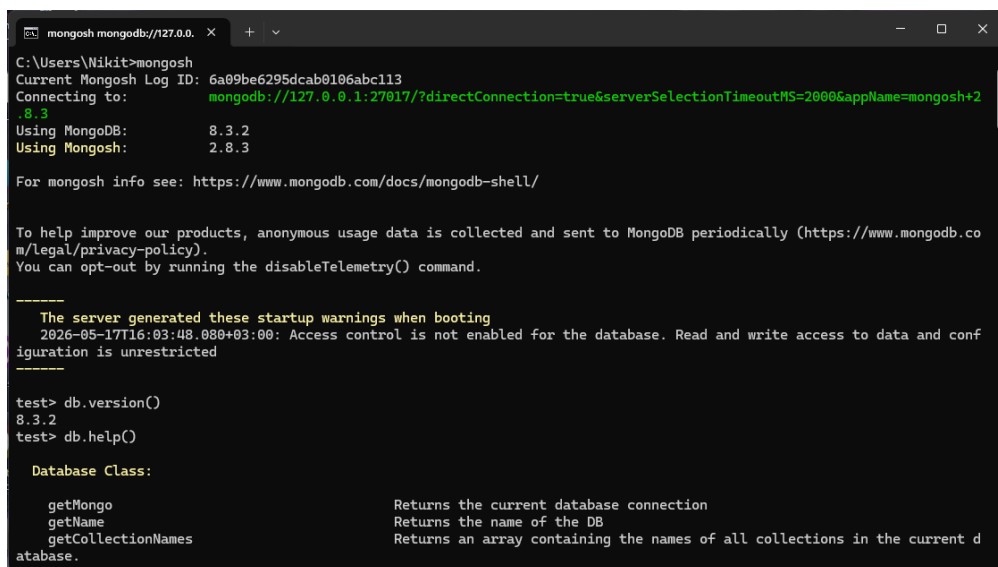
- `use <db>` — выбор базы данных;
- `show dbs` — просмотр списка баз данных;
- `show collections` — просмотр списка коллекций;
- `db.collection.insert()`, `insertMany()` — вставка документов;
- `db.collection.find()` — выборка документов;
- `sort()`, `limit()`, `skip()` — сортировка, ограничение и смещение выборки;
- `update()` — изменение документов;
- `remove()` — удаление документов;
- `ensureIndex()`, `getIndexes()`, `dropIndexes()` — создание, просмотр и удаление индексов.

3 Ход работы

3.1 Установка и проверка MongoDB

После установки MongoDB был запущен сервер базы данных и клиентская оболочка mongosh. Работоспособность проверялась командами `db.help()`, `db.help`, `db.stats()`, `db.version()`.

```
1 mongosh
2
3 db.version()
4 db.help()
5 db.help
6 db.stats()
```



```
mongosh mongodb://127.0.0.1
C:\Users\Nikit>mongosh
Current Mongosh Log ID: 6a09be6295dcab0106abc113
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB: 8.3.2
Using Mongosh: 2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

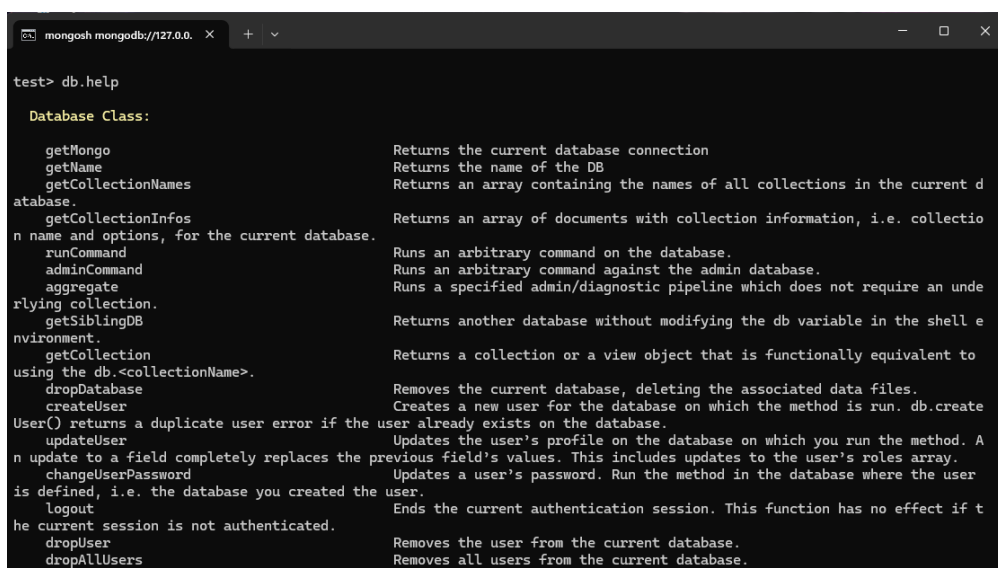
-----
The server generated these startup warnings when booting
2026-05-17T16:03:48.080+03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> db.version()
8.3.2
test> db.help()

Database Class:

getMongo           Returns the current database connection
getName            Returns the name of the DB
getCollectionNames Returns an array containing the names of all collections in the current database.
```

Рис. 1: Запуск MongoDB Shell, проверка версии, `db.help()`



```
test> db.help

Database Class:

getMongo           Returns the current database connection
getName            Returns the name of the DB
getCollectionNames Returns an array containing the names of all collections in the current database.
getCollectionInfos Returns an array of documents with collection information, i.e. collection name and options, for the current database.
runCommand         Runs an arbitrary command on the database.
adminCommand       Runs an arbitrary command against the admin database.
aggregate          Runs a specified admin/diagnostic pipeline which does not require an underlying collection.
getSiblingDB       Returns another database without modifying the db variable in the shell environment.
getCollection      Returns a collection or a view object that is functionally equivalent to using the db.<collectionName>.
dropDatabase       Removes the current database, deleting the associated data files.
createUser         Creates a new user for the database on which the method is run. db.createUser() returns a duplicate user error if the user already exists on the database.
updateUser         Updates the user's profile on the database on which you run the method. A n update to a field completely replaces the previous field's values. This includes updates to the user's roles array.
changeUserPassword Updates a user's password. Run the method in the database where the user is defined, i.e. the database you created the user.
logout             Ends the current authentication session. This function has no effect if the current session is not authenticated.
dropUser           Removes the user from the current database.
dropAllUsers       Removes all users from the current database.
```

Рис. 2: Выполнение `db.help`

```
mongosh mongodb://127.0.0.1:27020/
getLastError          Calls the getLastError command
printShardingStatus   Calls sh.status(verbose)
printSecondaryReplicationInfo Prints secondary replicaset information
getReplicationInfo    Returns replication information
printReplicationInfo  Formats sh.getReplicationInfo
printSlaveReplicationInfo DEPRECATED. Use db.printSecondaryReplicationInfo
setSecondaryOk        This method is deprecated. Use db.getMongo().setReadPref() instead
watch                 Opens a change stream cursor on the database
sql                   (Experimental) Runs a SQL query against Atlas Data Lake. Note: this is an
experimental feature that may be subject to change in future releases.
checkMetadataConsistency Returns a cursor with information about metadata inconsistencies

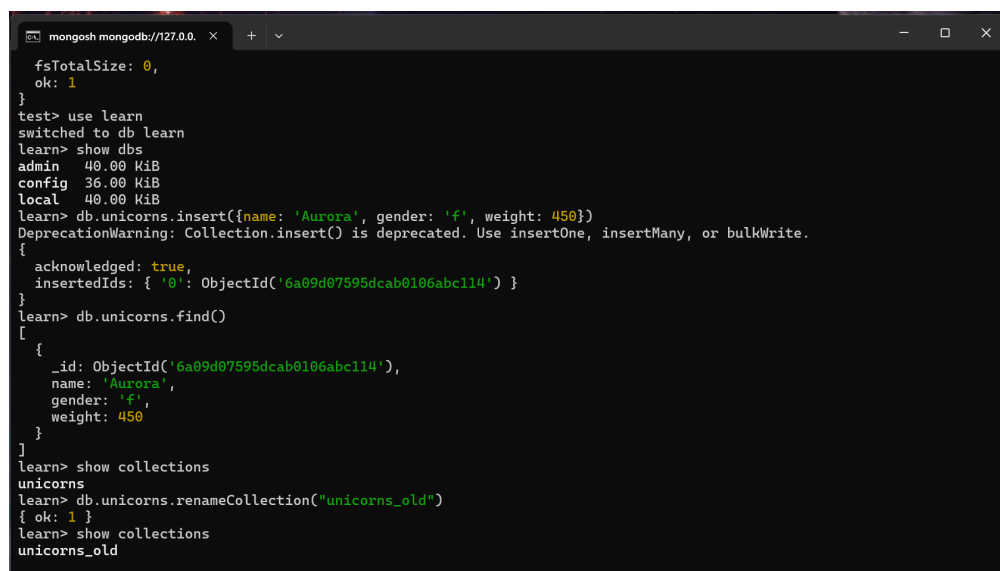
test> db.stats()
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
```

Рис. 3: Выполнение db.stats()

3.2 Создание и удаление базы данных в лабораторной работе 6.1

В MongoDB база данных создается автоматически при первой записи документа в любую ее коллекцию. Для выполнения задания была выбрана база данных **learn**, создана коллекция **unicorns**, после чего коллекция была переименована, просмотрена статистика, а затем коллекция и база данных были удалены.

```
1 use learn
2 show dbs
3
4 db.unicorns.insert({name: 'Aurora', gender: 'f', weight: 450})
5 show collections
6
7 db.unicorns.renameCollection('unicorns_old')
8 show collections
9
10 db.unicorns_old.stats()
11 db.unicorns_old.drop()
12 show collections
13
14 db.dropDatabase()
15 show dbs
```



```
mongosh mongod://127.0.0.1:27017
fsTotalSize: 0,
ok: 1
}
test> use learn
switched to db learn
learn> show dbs
admin    40.00 KiB
config   36.00 KiB
local    40.00 KiB
learn> db.unicorns.insert({name: 'Aurora', gender: 'f', weight: 450})
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '_id': ObjectId('6a09d07595dcab0106abc114') }
}
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a09d07595dcab0106abc114'),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  }
]
learn> show collections
unicorns
learn> db.unicorns.renameCollection("unicorns_old")
{ ok: 1 }
learn> show collections
unicorns_old
```

Рис. 4: Создание БД **learn** и коллекции **unicorns** с последующим переименованием

```

mongosh mongodb://127.0.0.1:27020
learn> db.unicorns_old.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4KB,app_metadata=(formatVersion=1),assert=(commit_time
stamp=none,durable_timestamp=none,read_timestamp=none,write_timestamp=off),block_allocation=best,block_compressor=sn
appy,block_manager=default,cache_resident=false,checksum=on,colgroups=,collator=,columns=,dictionary=0,disaggregated
=(page_log=,storage_tier=none),encryption=(keyid=,name=),exclusive=false,extractor=,format=btree,huffman_key=,huffma
n_value=,ignore_in_memory_cache_size=false,immutable=false,import=(compare_timestamp=oldest_timestamp,enabled=false,
file_metadata=,metadata_file=,panic_corrupt=true,repair=false),in_memory=false,ingest=,internal_item_max=0,internal_
key_max=0,internal_key_truncate=true,internal_page_max=4KB,key_format=q,key_gap=10,leaf_item_max=0,leaf_key_max=0,le
af_page_max=32KB,leaf_value_max=64MB,log=(enabled=true),lsm=(auto_throttle=,bloom=,bloom_bit_count=,bloom_config=,bl
oom_hash_count=,bloom_oldest=,chunk_count_limit=,chunk_max=,chunk_size=,merge_custom=(prefix=,start_generation=,suff
ix=),merge_max=,merge_min=),memory_page_image_max=0,memory_page_max=10M,os_cache_dirty_max=0,os_cache_max=0,prefix_c
ompression=false,prefix_compression_min=4,source=,split_deepen_min_child=0,split_deepen_per_child=0,split_pct=90,sta
ble=,tiered_storage=(auth_tokens=,bucket=,bucket_prefix=,cache_directory=,local_retention=300,name=,object_target_siz
e=0),type=file,value_format=u,verbose=[],write_timestamp_usage=none',
    type: 'file',
    uri: 'statistics:table:collection-089e2725-6457-4f0e-b6b5-0b7eb99fd93',
    version: 'WiredTiger 12.0.0: (November 15, 2024)',
    autocommit: {
      'retries for readonly operations': 0,
      'retries for update operations': 0
    },
    backup: {
      'total modified incremental blocks with compressed data': 0,
      'total modified incremental blocks without compressed data': 0
    },
  },
}

```

Рис. 5: Статистика коллекции unicorns

```

mongosh mongodb://127.0.0.1:27020
getLastError          Calls the getLastError command
printShardingStatus   Calls sh.status(verbose)
printSecondaryReplica Print secondary replicaset information
setReplicationInfo    Returns replication information
printReplicationInfo  Formats sh.getReplicationInfo
printSlaveReplicaInfo DEPRECATED. Use db.printSecondaryReplicationInfo
setSecondaryOk        This method is deprecated. Use db.getMongo().setReadPref() instead
watch                 Opens a change stream cursor on the database
sql                   (Experimental) Runs a SQL query against Atlas Data Lake. Note: this is an
experimental feature that may be subject to change in future releases.
checkMetadataConsistency Returns a cursor with information about metadata inconsistencies

test> db.stats()
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
test>

```

Рис. 6: Статистика коллекции unicorns

```
mongosh mongodb://127.0.0.1:27020
> use learn
switched to db learn
> db.unicorns_old.getIndexes()
{
  'rollback to stable updates from history store that would have been restored in non-dryrun mode': 0,
  'rollback to stable updates removed from history store': 0,
  'rollback to stable updates that would have been removed from history store in non-dryrun mode': 0,
  'update conflicts': 0
}
> db.unicorns_old.getShard()
{
  sharded: false,
  size: 65,
  count: 1,
  numOrphanDocs: 0,
  storageSize: 20480,
  totalIndexSize: 20480,
  totalSize: 40960,
  indexSizes: { _id_: 20480 },
  avgObjSize: 65,
  ns: 'learn.unicorns_old',
  nindexes: 1,
  scaleFactor: 1
}
> db.unicorns_old.drop()
true
> show collections
[]
> db.dropDatabase()
{ ok: 1, dropped: 'learn' }
> show dbs
admin 40.00 KiB
config 96.00 KiB
local 40.00 KiB
learn>
```

Рис. 7: Удаление коллекции unicorns и базы learn

3.3 Заполнение коллекции unicorns

Для дальнейших запросов была заново создана база данных learn и заполнена коллекция unicorns. Вставка выполнялась методом insertMany(), что является современным аналогом нескольких вызовов insert().

```
1 use learn
2
3 db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight:
4 600, gender: 'm', vampires: 63});
5 db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight:
6 450, gender: 'f', vampires: 43});
7 db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'],
8 weight: 984, gender: 'm', vampires: 182});
9 db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575,
10 gender: 'm', vampires: 99});
11 db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot',
12 'chocolate'], weight:550, gender:'f', vampires:80});
13 db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight:
14 733, gender: 'f', vampires: 40});
15 db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690,
gender: 'm', vampires: 39});
16 db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight:
421, gender: 'm', vampires: 2});
17 db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight:
601, gender: 'f', vampires: 33});
18 db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight:
650, gender: 'm', vampires: 54});
19 db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight:
540, gender: 'f'});
20
21 document = ({name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704,
gender: 'm', vampires: 165})
```



```

16 db.unicorns.insert(document)
17
18 db.unicorns.find()

```

```

mongosh mongodb://127.0.0.1
learn> db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63});
learn> db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
learn> db.unicorns.insert({name: 'Unicrom', loves: ['energy', 'redbull'], weight: 984, gender: 'm', vampires: 182});
learn> db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
learn> db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});

learn> db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
learn> db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
learn> db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
learn> db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
learn> db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
learn> db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});

{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a09d95395dcab0106abc11f') }
}
learn> document = ({name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165})
learn> db.unicorns.insert(document)

{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a09d99295dcab0106abc120') }
}
learn> db.unicorns.find()

[
  {
    _id: ObjectId('6a09d95395dcab0106abc115'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc117'),
    name: 'Unicrom',

```

Рис. 8: Заполнение коллекции unicorns

3.4 Выборка данных

Были сформированы запросы для получения самцов и самок единорогов, фильтрации по предпочтениям, ограничения количества документов, сортировки и проекции отдельных полей.

```

1 // Самцы по имени
2 db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(3)
3
4 // Самки по имени
5 db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
6
7 // Самки, которые любят carrot: findOne
8 db.unicorns.findOne({gender: 'f', loves: 'carrot'})
9
10 // Самки, которые любят carrot: find + limit
11 db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
12
13 // Самцы без loves и gender
14 db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
15

```

```

16 // Обратный порядок добавления
17 db.unicorns.find().sort({$natural: -1})
18
19 // Первый элемент массива loves без _id
20 db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})

```

The screenshot shows a MongoDB terminal window with the following command and output:

```

learn> db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a09d99295dcab0106abc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc115'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc11b'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]

```

Рис. 9: Самцы по имени

The screenshot shows a MongoDB terminal window with the following command and output:

```

learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a09d95395dcab0106abc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc11a'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc11d'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]

```

Рис. 10: Самки по имени

```
mongosh mongodb://127.0.0.1:27027/learn
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a09d95395dcab0106abc116'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a09d95395dcab0106abc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Рис. 11: Самки, которые любят carrot с findOne и limit

```
mongosh mongodb://127.0.0.1:27027/learn
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
[
  {
    _id: ObjectId('6a09d95395dcab0106abc115'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc117'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc118'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc11b'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc11c'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc11e'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a09d99295dcab0106abc120'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  }
]
```

Рис. 12: Самцы без loves и gender

```
mongosh mongodb://127.0.0.1
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a09d99295dcab0106abc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc11f'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc11e'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc11d'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc11c'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc11b'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
```

Рис. 13: Обратный порядок добавления

```
mongosh mongodb://127.0.0.1
learn> db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn> |
```

Рис. 14: Первый элемент массива loves без *id*

3.5 Логические операторы

Для отбора документов использовались операторы сравнения `$gte`, `$lte`, проверка существования поля `$exists`, а также оператор `$all` для поиска документов, массив которых содержит все указанные значения.

```

1 // Самки весом от 500 до 700 кг без _id
2 db.unicorns.find(
3   {gender: 'f', weight: {$gte: 500, $lte: 700}},
4   {_id: 0}
5 )
6
7 // Самцы весом от 500 кг, предпочитающие grape и lemon
8 db.unicorns.find(
9   {gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}},
10  {_id: 0}
11 )
12
13 // Единороги без поля vampires
14 db.unicorns.find({vampires: {$exists: false}})
15
16 // Упорядоченный список имен самцов и первое предпочтение
17 db.unicorns.find(
18   {gender: 'm'},
19   {_id: 0, name: 1, loves: {$slice: 1}}
20 ).sort({name: 1})

```

```

learn> db.unicorns.find(
|   {gender: 'f', weight: {$gte: 500, $lte: 700}},
|   {_id: 0}
| )
[
|   {
|     name: 'Solnara',
|     loves: [ 'apple', 'carrot', 'chocolate' ],
|     weight: 550,
|     gender: 'f',
|     vampires: 80
|   },
|   {
|     name: 'Leia',
|     loves: [ 'apple', 'watermelon' ],
|     weight: 601,
|     gender: 'f',
|     vampires: 33
|   },
|   {
|     name: 'Nimue',
|     loves: [ 'grape', 'carrot' ],
|     weight: 540,
|     gender: 'f'
|   }
| ]
learn> db.unicorns.find(
|   {gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}},
|   {_id: 0}
| )
[
|   {
|     name: 'Kenny',
|     loves: [ 'grape', 'lemon' ],
|     weight: 690,
|     gender: 'm',
|     vampires: 39
|   }
| ]

```

Рис. 15: Первые 3 запроса

```
mongosh mongodb://127.0.0.1:27027/learn
learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a09d95395dcab0106abc11f'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> db.unicorns.find(
  {
    {gender: 'm'},
    {_id: 0, name: 1, loves: {$slice: 1}}
  },
  {name: 1}
)
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
learn>
```

Рис. 16: Заключительные 2 запроса

3.6 Работа с вложенными документами

Была создана коллекция `towns`, в которой поле `mayor` является вложенным документом. Доступ к полям вложенного документа осуществляется через точечную нотацию, например `mayor.party`.

```
1 db.towns.insertMany([
2   {
3     name: 'Punxsutawney',
4     population: 6200,
5     last_sensus: ISODate('2008-01-31'),
6     famous_for: ['phil the groundhog'],
7     mayor: {name: 'Jim Wehrle'}
8   },
9   {
10    name: 'New York',
11    population: 22200000,
12    last_sensus: ISODate('2009-07-31'),
13    famous_for: ['statue of liberty', 'food'],
14    mayor: {name: 'Michael Bloomberg', party: 'I'}
15  },
16  {
17    name: 'Portland',
18    population: 528000,
19    last_sensus: ISODate('2009-07-20'),
20    famous_for: ['beer', 'food'],
21    mayor: {name: 'Sam Adams', party: 'D'}
22  }
23 ])
24
25 // Города с независимыми мэрами
26 db.towns.find({'mayor.party': 'I'}, {_id: 0, name: 1, mayor: 1})
```

```

27
28 // Города с беспартийными мэрами
29 db.towns.find({'mayor.party': {$exists: false}}, {_id: 0, name: 1, mayor:
1})

```

```

learn> db.towns.insertMany([
  {
    name: 'Punxsutawney',
    population: 6200,
    last_sensus: ISODate('2008-01-31'),
    famous_for: ['phil the groundhog'],
    mayor: {name: 'Jim Wehrle'}
  },
  {
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31'),
    famous_for: ['statue of liberty', 'food'],
    mayor: {name: 'Michael Bloomberg', party: 'I'}
  },
  {
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20'),
    famous_for: ['beer', 'food'],
    mayor: {name: 'Sam Adams', party: 'D'}
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a09f60b95dcab0106abc121'),
    '1': ObjectId('6a09f60b95dcab0106abc122'),
    '2': ObjectId('6a09f60b95dcab0106abc123')
  }
}

```

Рис. 17: Создание и вставка в коллекцию towns

```

learn> db.towns.find({'mayor.party': 'I'}, {_id: 0, name: 1, mayor: 1})
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
learn> db.towns.find({'mayor.party': {$exists: false}}, {_id: 0, name: 1, mayor: 1})
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn>

```

Рис. 18: Запросы к коллекции towns

3.7 Использование JavaScript и курсоров

В оболочке MongoDB можно использовать JavaScript-функции и курсоры. Курсор представляет результат выборки и позволяет поэлементно обрабатывать документы.

```

1 let cursor = db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(2)
2
3 cursor.forEach(function(obj) {
4   print(obj.name + ' | weight=' + obj.weight + ' | loves=' + obj.loves[0])
5 })

```

```
mongosh mongodb://127.0.0.1
learn> let cursor = db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(2)

learn> cursor.forEach(function(obj) {
|   print(obj.name + ' | weight=' + obj.weight + ' | loves=' + obj.loves[0])
| })
Dunx | weight=704 | loves=grape
Horny | weight=600 | loves=carrot

learn> |
```

Рис. 19: Использование курсора и forEach

3.8 Агрегированные запросы

Были выполнены подсчет количества документов, получение уникальных значений массива `loves`, а также группировка по полу.

```
1 // Количество самок весом от 500 до 600 кг
2 db.unicorns.find({
3   gender: 'f',
4   weight: {$gte: 500, $lte: 600}
5 }).count()
6
7 // Список уникальных предпочтений
8 db.unicorns.distinct("loves")
9
10 // Количество особей обоих полов
11 db.unicorns.aggregate({
12   "$group": {
13     _id: "$gender",
14     count: {$sum: 1}
15   }
16 })
```

```
mongosh mongodb://127.0.0.1
learn> db.unicorns.find({
|   gender: 'f',
|   weight: {$gte: 500, $lte: 600}
| }).count()
2
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn> db.unicorns.aggregate({
|   "$group": {
|     _id: "$gender",
|     count: {$sum: 1}
|   }
| })
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
learn> |
```

Рис. 20: Агрегированные запросы

3.9 Редактирование документов

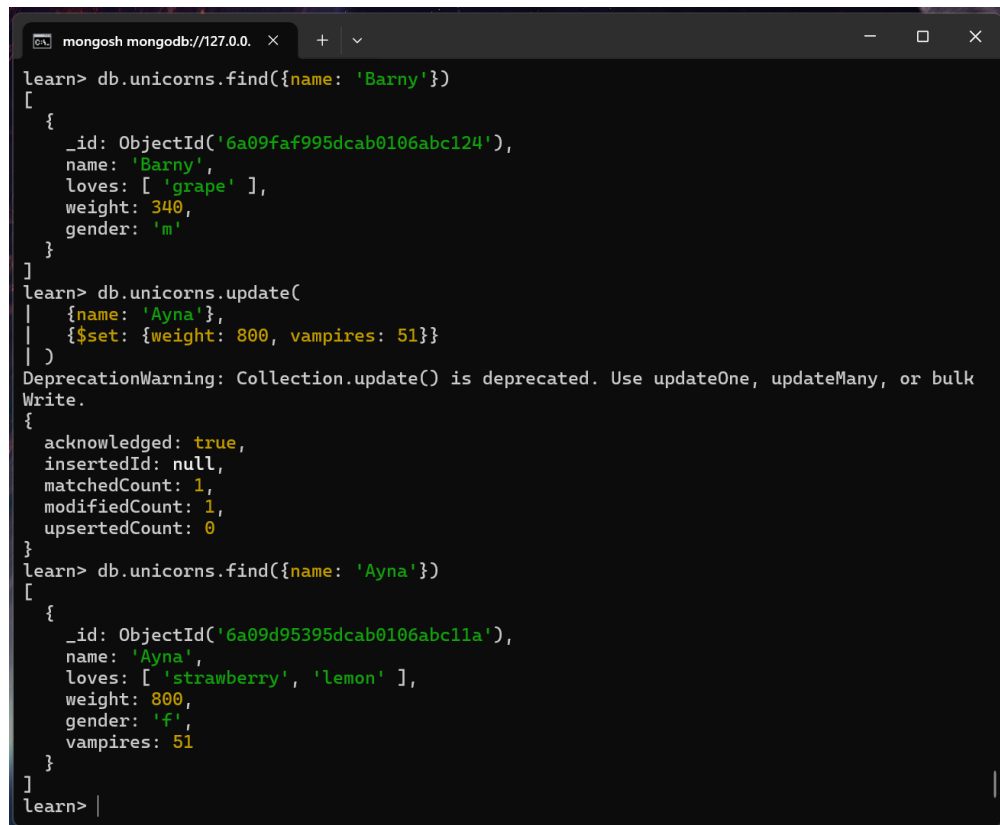
Для изменения документов применялись операторы `$set`, `$inc`, `$unset`, `$push`, `$addToSet`. Они позволяют изменять отдельные поля документа без полной перезаписи документа.

```
1 // Добавление Barny
2 db.unicorns.save({name: 'Barny', loves: ['grape'], weight: 340, gender:
  'm'})
3
4 // Проверка
5 db.unicorns.find({name: 'Barny'})
6
7 // Ayna: вес 800, vampires 51
8 db.unicorns.update(
9   {name: 'Ayna'},
10  {$set: {weight: 800, vampires: 51}}
11 )
12
13 // Проверка
14 db.unicorns.find({name: 'Ayna'})
15
16 // Raleigh теперь любит redbull
17 db.unicorns.update(
18   {name: 'Raleigh'},
19   {$set: {loves: ['redbull']}}
20 )
21
22 // Проверка
23 db.unicorns.find({name: 'Raleigh'})
24
25 // Всем самцам увеличить vampires на 5
26 db.unicorns.update(
27   {gender: 'm'},
28   {$inc: {vampires: 5}},
29   {multi: true}
30 )
31
32 // Проверка
33 db.unicorns.find({gender: 'm'})
34
35 // Pilot теперь любит chocolate
36 db.unicorns.update(
37   {name: 'Pilot'},
38   {$push: {loves: 'chocolate'}}
39 )
40
41 // Проверка
42 db.unicorns.find({name: 'Pilot'})
43
44 // Aurora теперь любит sugar и lemon, без дублей
45 db.unicorns.update(
```

```

46 {name: 'Aurora'},
47 {$addToSet: {loves: {$each: ['sugar', 'lemon']}}}
48 )
49
50 // Проверка
51 db.unicorns.find({name: 'Aurora'})

```



```

mongosh mongod://127.0.0.1
learn> db.unicorns.find({name: 'Barney'})
[
  {
    _id: ObjectId('6a09faf995dcab0106abc124'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
learn> db.unicorns.update(
  {name: 'Ayna'},
  {$set: {weight: 800, vampires: 51}}
)
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a09d95395dcab0106abc11a'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
learn>

```

Рис. 21: Работа с update в unicorns: 1-2 запросы

```
mongosh mongodb://127.0.0.1:27020/learn > db.unicorns.update(
  {
    name: 'Raleigh',
    $set: {loves: ['redbull']}
  },
  {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
)
learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a09d95395dcab0106abc11c'),
    name: 'Raleigh',
    loves: [ 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
learn> db.unicorns.update(
  {
    gender: 'm',
    $inc: {vampires: 5},
    $multi: true
  },
  {
    acknowledged: true,
    insertedId: null,
    matchedCount: 8,
    modifiedCount: 8,
    upsertedCount: 0
  }
)
learn> db.unicorns.find({gender: 'm'})
[
  {
    _id: ObjectId('6a09d95395dcab0106abc115'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('6a09d95395dcab0106abc11c'),
    name: 'Raleigh',
    loves: [ 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 7
  }
]
```

Рис. 22: Работа с update в unicorns: 3-4 запросы

```
mongosh mongodb://127.0.0.1:27021/learn
learn> db.unicorns.update(
  {
    name: 'Pilot',
    loves: { $push: { loves: 'chocolate' } }
  },
  {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
)
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a09d95395dcab0106abc11e'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn> db.unicorns.update(
  {
    name: 'Aurora',
    loves: { $addToSet: { loves: { $each: [ 'sugar', 'lemon' ] } } }
  },
  {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
)
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a09d95395dcab0106abc116'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Рис. 23: Работа с update в unicorns: 5-6 запросы

```
1 // Portland: удалить party у mayor
2 db.towns.update(
3   {name: 'Portland'},
4   { $unset: { "mayor.party": 1 } }
5 )
6
7 // Проверка
8 db.towns.find({name: 'Portland'})
```

```
mongosh mongodb://127.0.0.1:27020/learn > db.towns.update(
  {
    name: 'Portland',
    $unset: {mayor.party: 1}
  },
  {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
)
learn> db.towns.find({name: 'Portland'})
[
  {
    _id: ObjectId('6a09f60b95dcab0106abc123'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
```

Рис. 24: Работа с update в towns

3.10 Удаление документов

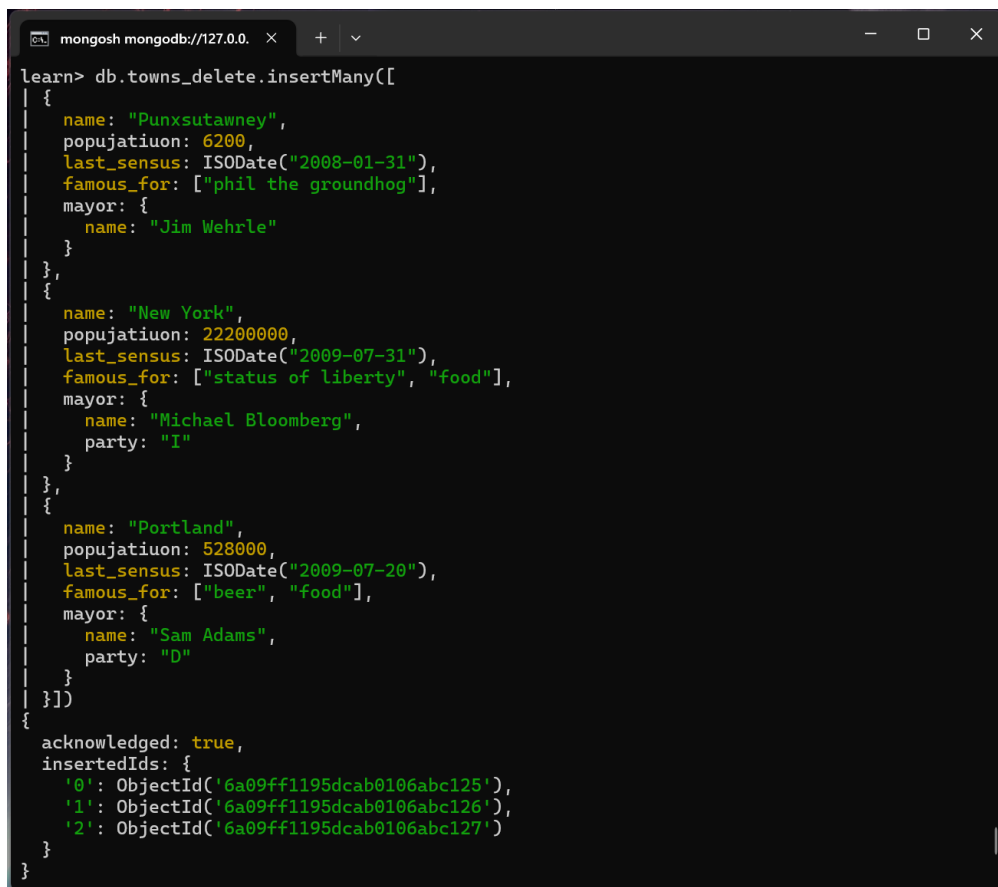
В MongoDB удаление документов выполняется по условию. В данной работе из коллекции `towns` были удалены документы с беспартийными мэрами, после чего коллекция была очищена.

```
1 // Создание коллекции для работы с удалением
2 db.towns_delete.insertMany([
3   {
4     name: "Punxsutawney",
5     popujatiuon: 6200,
6     last_sensus: ISODate("2008-01-31"),
7     famous_for: ["phil the groundhog"],
8     mayor: {
9       name: "Jim Wehrle"
10    }
11  },
12  {
13    name: "New York",
14    popujatiuon: 22200000,
15    last_sensus: ISODate("2009-07-31"),
16    famous_for: ["status of liberty", "food"],
17    mayor: {
18      name: "Michael Bloomberg",
19      party: "I"
20    }
21  },
22  {
23    name: "Portland",
24    popujatiuon: 528000,
25    last_sensus: ISODate("2009-07-20"),
26    famous_for: ["beer", "food"],
27    mayor: {
```

```

28     name: "Sam Adams",
29     party: "D"
30   }
31 })
32
33 // Удалить города с беспартийными мэрами
34 db.towns_delete.remove({"mayor.party": {$exists: false}})
35 db.towns_delete.find()
36
37 // Очистить коллекцию
38 db.towns_delete.remove({})
39 db.towns_delete.find()
40
41 show collections

```



```

mongosh mongodb://127.0.0.1
learn> db.towns_delete.insertMany([
  {
    name: "Punxsutawney",
    popujatiuon: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {
    name: "New York",
    popujatiuon: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"
    }
  },
  {
    name: "Portland",
    popujatiuon: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"
    }
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a09ff1195dcab0106abc125'),
    '1': ObjectId('6a09ff1195dcab0106abc126'),
    '2': ObjectId('6a09ff1195dcab0106abc127')
  }
}

```

Рис. 25: Создание коллекции *towns_delete*

```
mongosh mongodb://127.0.0.1:27021/learn
learn> db.towns_delete.remove({"mayor.party": {$exists: false}})
DeprecationWarning: Collection.remove() is deprecated. Use deleteOne, deleteMany, findOne
AndDelete, or bulkWrite.
{ acknowledged: true, deletedCount: 1 }
learn> db.towns_delete.find()
[
  {
    _id: ObjectId('6a09ff1195dcab0106abc126'),
    name: 'New York',
    popujatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a09ff1195dcab0106abc127'),
    name: 'Portland',
    popujatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn> db.towns_delete.remove({})
{ acknowledged: true, deletedCount: 2 }
learn> db.towns_delete.find()
[]
learn> show collections
towns
towns_delete
unicorns
learn>
```

Рис. 26: Работа с remove

3.11 Ссылки между документами

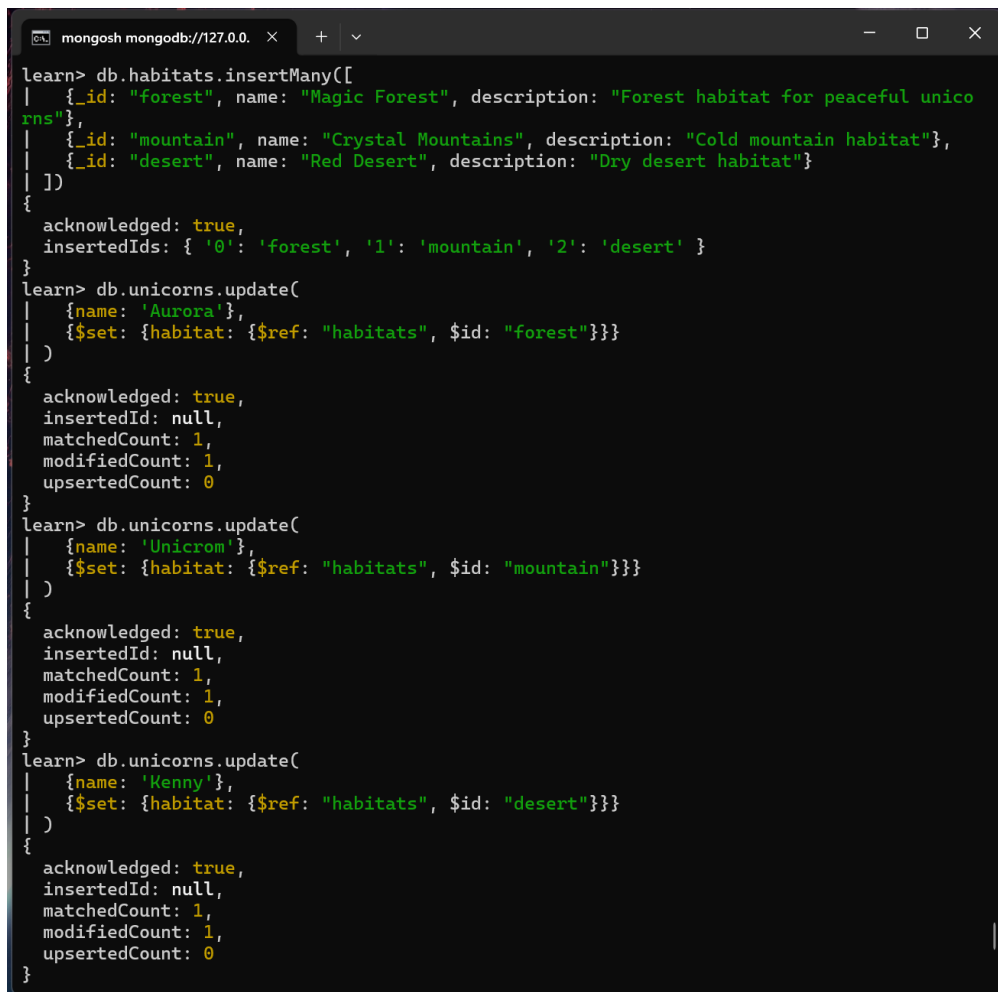
Была создана коллекция зон обитания единорогов `habitats`. В документы нескольких единорогов добавлена ссылка на зону обитания в формате `{$ref, $id}`.

```
1 // Создать коллекцию зон обитания
2 db.habitats.insertMany([
3   { _id: "forest", name: "Magic Forest", description: "Forest habitat for
4     peaceful unicorns" },
5   { _id: "mountain", name: "Crystal Mountains", description: "Cold mountain
6     habitat" },
7   { _id: "desert", name: "Red Desert", description: "Dry desert habitat" }
8 ])
9
10 // Добавить ссылки нескольким единорогам
11 db.unicorns.update(
12   { name: 'Aurora' },
13   { $set: { habitat: { $ref: "habitats", $id: "forest" } } }
14 )
15 db.unicorns.update(
16   { name: 'Unicrom' },
17   { $set: { habitat: { $ref: "habitats", $id: "mountain" } } }
18 )
19 db.unicorns.update(
20   { name: 'Kenny' },
21   { $set: { habitat: { $ref: "habitats", $id: "desert" } } }
22 )
```

```

21
22 // Проверить содержание коллекции единорогов
23 db.unicorns.find(
24   {habitat: {$exists: true}},
25   {_id: 0, name: 1, habitat: 1}
26 )
27
28 // Проверить переход по ссылке
29 var aurora = db.unicorns.findOne({name: 'Aurora'})
30 db[aurora.habitat.collection].findOne({_id: aurora.habitat.oid})

```



```

mongosh mongodb://127.0.0.1
learn> db.habitats.insertMany([
  |   {_id: "forest", name: "Magic Forest", description: "Forest habitat for peaceful unicorns"},
  |   {_id: "mountain", name: "Crystal Mountains", description: "Cold mountain habitat"},
  |   {_id: "desert", name: "Red Desert", description: "Dry desert habitat"}
  | ])
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'mountain', '2': 'desert' }
}
learn> db.unicorns.update(
  |   {name: 'Aurora'},
  |   {$set: {habitat: {$ref: "habitats", $id: "forest"}}}
  | )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.update(
  |   {name: 'Unicrom'},
  |   {$set: {habitat: {$ref: "habitats", $id: "mountain"}}}
  | )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.update(
  |   {name: 'Kenny'},
  |   {$set: {habitat: {$ref: "habitats", $id: "desert"}}}
  | )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

Рис. 27: Создание коллекции и ссылок


```
mongosh mongodb://127.0.0.1:27021/learn
learn> db.unicorns.find(
  {habitat: {$exists: true}},
  {_id: 0, name: 1, habitat: 1}
)
[
  { name: 'Aurora', habitat: DBRef('habitats', 'forest') },
  { name: 'Unicrom', habitat: DBRef('habitats', 'mountain') },
  { name: 'Kenny', habitat: DBRef('habitats', 'desert') }
]
learn> var aurora = db.unicorns.findOne({name: 'Aurora'})
learn> db[aurora.habitat.collection].findOne({_id: aurora.habitat.oid})
{
  _id: 'forest',
  name: 'Magic Forest',
  description: 'Forest habitat for peaceful unicorns'
}
learn> |
```

Рис. 28: Проверка ссылок и содержания коллекции

3.12 Индексы

Индексы ускоряют поиск по выбранным полям. Был создан уникальный индекс по полю **name** коллекции **unicorns**. Так как имена единорогов в коллекции уникальны, индекс создается успешно. Если попытаться вставить еще один документ с уже существующим именем, будет получена ошибка нарушения уникальности.

```
1 // Создание уникального индекса по имени
2 db.unicorns.ensureIndex({"name": 1}, {"unique": true})
3
4 // Проверка списка индексов
5 db.unicorns.getIndexes()
6
7 // Удалить все индексы, кроме индекса для идентификатора
8 db.unicorns.dropIndexes()
9
10 // Проверка
11 db.unicorns.getIndexes()
12
13 // Попытаться удалить индекс для идентификатора
14 db.unicorns.dropIndex("_id_")
```

```
mongosh mongodb://127.0.0.1:27020/learn
learn> db.unicorns.ensureIndex({"name": 1}, {"unique": true})
[ 'name_1' ]
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.dropIndexes()
{
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn>
```

Рис. 29: Работа с индексами

3.13 План запроса

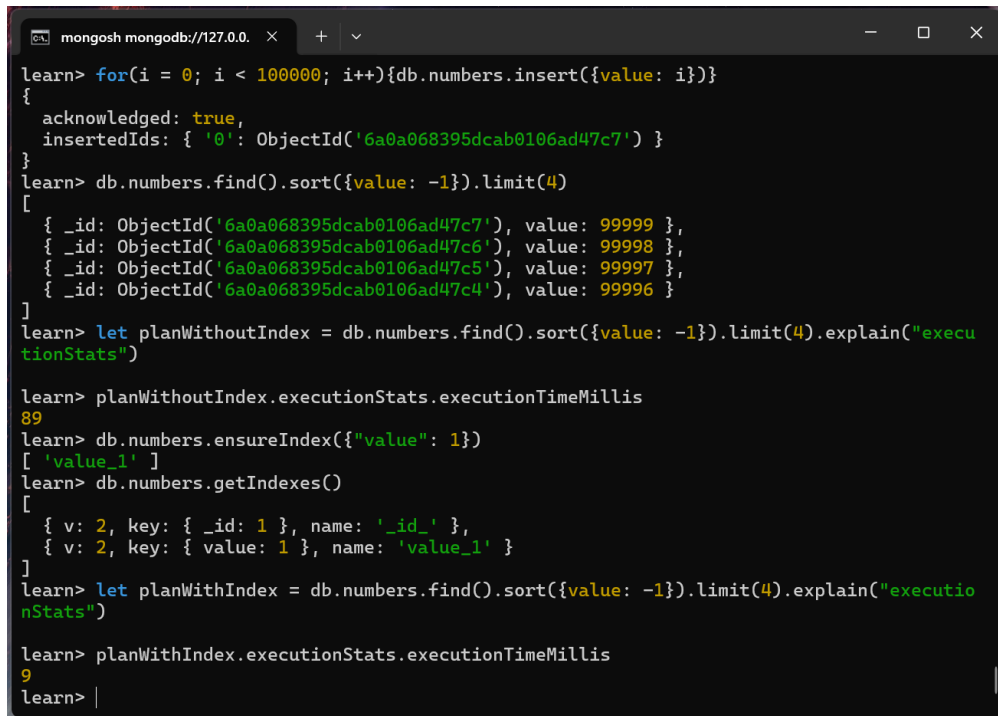
Для анализа плана запроса была создана коллекция **numbers**, содержащая 100000 документов. Затем был выполнен запрос на выбор последних четырех документов по значению поля **value**.

С помощью метода `explain("executionStats")` было определено время выполнения запроса до создания индекса и после создания индекса.

```
1 // Создать объемную коллекцию numbers
2 for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
3
4 // Выбрать последние четыре документа
5 db.numbers.find().sort({value: -1}).limit(4)
6
7 // Проанализировать план запроса без индекса
8 let planWithoutIndex = db.numbers.find().sort({value:
9 -1}).limit(4).explain("executionStats")
10 planWithoutIndex.executionStats.executionTimeMillis
11
12 // Создать индекс для ключа value
13 db.numbers.ensureIndex({"value": 1})
14
15 // Получить информацию обо всех индексах коллекции numbers
16 db.numbers.getIndexes()
17
18 // Выполнить запрос 2 повторно и проанализировать
19 let planWithIndex = db.numbers.find().sort({value:
20 -1}).limit(4).explain("executionStats")
21 planWithIndex.executionStats.executionTimeMillis
```

До создания индекса по полю **value** время выполнения запроса составило 89 мс. После создания индекса время выполнения запроса составило 9 мс. Следовательно, запрос с индексом является более эффективным, так

как MongoDB использует индекс для быстрого доступа к нужным документам и не выполняет полный перебор коллекции.



```
learn> for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a0a068395dcab0106ad47c7') }
}
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a0a068395dcab0106ad47c7'), value: 99999 },
  { _id: ObjectId('6a0a068395dcab0106ad47c6'), value: 99998 },
  { _id: ObjectId('6a0a068395dcab0106ad47c5'), value: 99997 },
  { _id: ObjectId('6a0a068395dcab0106ad47c4'), value: 99996 }
]
learn> let planWithoutIndex = db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")

learn> planWithoutIndex.executionStats.executionTimeMillis
89
learn> db.numbers.ensureIndex({"value": 1})
[ 'value_1' ]
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn> let planWithIndex = db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")

learn> planWithIndex.executionStats.executionTimeMillis
9
learn> |
```

Рис. 30: Работа с executionStats

4 Ответы на контрольные вопросы

Лабораторная работа 6.1

1. Какую модель данных поддерживает MongoDB? MongoDB поддерживает документо-ориентированную модель данных. Вместо таблиц и строк, как в реляционных базах данных, данные хранятся в виде документов, состоящих из пар “ключ–значение”.

2. Какой формат данных поддерживается в MongoDB? MongoDB использует формат BSON, то есть Binary JSON. По структуре он похож на JSON, но хранится в бинарном виде, что позволяет быстрее обрабатывать данные.

3. Из чего состоит база данных MongoDB? База данных MongoDB состоит из коллекций, а коллекции состоят из документов. Документ содержит поля и значения, причем значения могут быть простыми типами, массивами или вложенными объектами.

4. Имеет ли БД MongoDB жесткую структуру? Нет, MongoDB не имеет жесткой схемы данных. В одной коллекции могут храниться документы с разным набором полей и разной структурой. Если какое-то поле не нужно, оно просто не добавляется в документ.

Лабораторная работа 6.2

1. Как используется оператор точка? Оператор точка используется для обращения к полям вложенных объектов. Например, если в документе есть объект `mayor`, а внутри него поле `party`, то запрос к этому полю записывается как `"mayor.party"`. Это позволяет искать документы не только по обычным полям, но и по значениям внутри вложенных структур.

2. Как можно использовать курсор? Курсор используется для последовательной обработки результатов запроса. Метод `find()` возвращает не сразу обычный массив, а курсор, по которому можно проходить с помощью `hasNext()`, `next()` или `forEach()`. Также к курсору можно применять `sort()`, `limit()` и `skip()`.

3. Какие возможности агрегирования данных существуют в MongoDB? В MongoDB можно выполнять простые агрегированные операции, например подсчет количества документов через `count()`, получение уникальных значений через `distinct()`, а также более сложную обработку данных через `aggregate()`. Метод `aggregate()` позволяет группировать документы, считать количество элементов в группах и выполнять обработку, похожую на `GROUP BY` в SQL.

4. Какая из функций `save` или `update` более детально позволяет настроить редактирование документов коллекции? Более детально редактирование позволяет настроить функция `update`. Она дает возможность задать условие выбора документа, использовать операторы изменения, например `$set`, `$inc`, `$unset`, а также управлять параметрами обновления, например обновлять один документ или сразу несколько.

5. Как происходит удаление документов из коллекции по умолчанию? Удаление выполняется с помощью метода `remove()`, которому передается условие отбора документов. По умолчанию удаляются все документы, которые подходят под заданное условие. Если передать пустое условие `{}`, то будут удалены все документы из коллекции.

6. Назовите способы связывания коллекций в MongoDB. В MongoDB коллекции можно связывать двумя способами.

Первый способ – ручная установка ссылки. В этом случае в одном документе сохраняется значение поля `_id` другого документа. Например, в документе пользователя можно хранить поле `company`, в котором записан идентификатор компании из коллекции `companies`.

Второй способ – автоматическое связывание с использованием `DBRef`. В этом случае ссылка хранится в виде структуры с полями `$ref` и `$id`, где `$ref` содержит имя коллекции, а `$id` – идентификатор связанного документа. При необходимости также может указываться поле `$db` с именем базы данных.

При этом MongoDB не создает внешние ключи как в реляционных СУБД, поэтому связь между документами является логической и обра-

батывается на уровне запросов или приложения.

7. Сколько индексов можно установить на одну коллекцию?

MongoDB позволяет установить до 64 индексов на одну коллекцию.

8. Как получить информацию о всех индексах базы данных MongoDB? Информацию об индексах можно получить через системную коллекцию `system.indexes`, а для отдельной коллекции — методом `getIndexes()`, например `db.unicorns.getIndexes()`.

5 Вывод

В ходе лабораторной работы были изучены базовые приемы работы с MongoDB: запуск СУБД, создание базы данных и коллекций, выполнение CRUD-операций, запросов, агрегации, работы со ссылками и индексами. На практике было показано, что MongoDB позволяет гибко хранить и обрабатывать документы разной структуры без жесткой схемы данных.